

面向高频 DApp 的 Hybrid TEE-Rollup 可恢复挑战协议与轻量 DA 成本评估

Recoverable Challenge Protocol and Evidence-Carrying DA Commitments for Hybrid TEE-Rollups in High-Frequency DApps

作者：杨帆

摘要

高频 DApp，如元宇宙状态同步、去中心化社交交互和链上 AI 推理服务，要求系统同时具备低交互成本、较低延迟和异常情况下的可验证性。全链上执行虽然可验证性强，但难以承受高频负载；完全链下执行虽能降低成本，却削弱公开可验证与可追责能力；单纯依赖 TEE 又会引入硬件信任与可用性风险。围绕这一矛盾，本文在 Hybrid TEE-Rollup 的设计空间中研究一条更窄但关键的异常路径：当提交者给出轻量链上承诺、完整 payload 位于 DA 层、挑战过程又因 timeout 停滞时，系统如何恢复争议、重放证据并完成仲裁。论文将 recoverable challenge、evidence-carrying compact commit 和 DA-aware cost model 置于同一研究原型中，并通过 Python prototype、本地 EVM gasUsed 与 Sepolia 部署记录给出阶段性证据。实验显示，compact commit 的平均规模约为 406 bytes；在给定 cost model 与 DA profile 下，compact modular DA sampling 相比 full-onchain calldata 在 payload=8192、prompt length=64、batch size=1000 时可得到 96.33% 的估算降本；recoverable challenge 在 timeout 场景下将 challenge success 从 0% 提升至 100%。这些结果说明了机制接口与成本趋势，而不外推为真实 TEE 安全、生产级 DA 或主网性能结论。

关键词：Hybrid TEE-Rollup；交互式挑战；数据可用性；高频 DApp；成本摊销；故障恢复

1 Introduction

高频 DApp 把区块链扩容问题推到一个不容易回避的位置：单次交互价值往往不高，但状态更新连续、请求密度高，且异常结果仍需要公开可追责。以一个链上 AI 推理服务为例，用户连续提交 prompt，执行者在 TEE 中生成 response，并向链上提交 compact commit；完整 prompt、response 与 attestation 被放入 DA 层。正常情况下，这条路径只需要轻量提交。但一旦执行者提交的 response 与 DA payload 不一致，或在 challenge 中途超时失联，系统就不能只回答“是否检测到异常”，还必须回答争议是否能继续推进到 replay 与 resolve。

现有 Rollup 路线为这一张力提供了不同折中。ZK-Rollup 依靠有效性证明获得强验证属性，但证明生成、系统工程和电路设计成本并不适合所有高频交互；Optimistic Rollup 将常规路径做轻，却把延迟和复杂性推到挑战窗口与 fraud proof 流程中；TEE-Rollup 或 Hybrid TEE-Rollup 借助可信执行环境加速链下执行，再通过链上提交与挑战机制保留异常处理入口。问题在于，TEE 不能被当作绝对可信组件：硬件漏洞、宿主环境、I/O 操控、远程证明失效和服务不可用，都可能让“链下执行很快”与“异常时可验证”之间出现断点。

这类断点在 timeout 场景中尤其明显。传统 challenge 机制通常强调错误能否被发现，但高频 DApp 还要求争议流程具有活性：如果 challenge session 停在中间状态，检测到异常并不能完成止损，错误提交仍可能占据未决状态。本文因此把 failure -> recovery -> replay ->

resolution 作为贯穿全文的研究主线：异常被观察后，协议需要恢复会话；恢复后，证据需要支持 replay；replay 后，仲裁结果需要可结算。

轻量提交也存在类似边界。compact commit 如果只是一个孤立哈希，确实能减少链上字节数，却不足以支撑后续验证；它必须回指 DA payload、字段级 Merkle proof 与执行摘要，才能在争议发生时成为 evidence entry。本文将正常路径与异常路径拆开：正常路径追求低链上提交成本，异常路径要求 challenge 能够恢复推进，并且能够从 DA 结构中取回可检查证据。实验与实现部分由三类证据组成：Python 原型验证状态机和成本趋势，本地 EVM gasUsed 观察合约关键路径开销，Sepolia 部署记录说明核心合约具备公开测试网可部署性。

1.1 Contributions（本文贡献）

本文的第一项贡献是将 Hybrid TEE-Rollup 中的 challenge 从一次性错误检测流程扩展为具备恢复语义的状态机。当 challenge session 因 timeout 或中断停滞时，watchdog/operator 可以触发 recover，使会话继续推进至 replay 与 resolve；recover 不改写争议事实，而是恢复协议活性。

第二项贡献是 evidence-carrying compact commit。该机制把 state root、output hash、proof hash、DA root 与 payload proof 绑定在同一提交语境中，使 compact commit 不只是节省链上空间的摘要，而是后续验证者进入 DA 证据结构的最小入口。

第三项贡献是 DA-aware cost model。该模型联合 payload size、batch size 与 DA route 解释轻量提交的成本边界，并将成本收益放回异常可验证性的系统权衡中。三项机制共同服务于同一问题：在高频 DApp 中，如何用较低链上负担保留异常时的可验证性、可恢复性与成本解释能力。

表 1 本文机制推进点与研究作用

机制推进	核心作用	论文定位
Recoverable Challenge	在 timeout 或中断后恢复争议流程	提升 challenge liveness
Evidence-Carrying Compact Commit	绑定 state/output/proof/DA root	形成最小证据入口
Fault Taxonomy	区分 attestation、DA、timeout、replay 和 trace 故障	明确异常责任边界
DA-aware Cost Amortization	联合 payload、batch 和 DA route 分析成本	支撑高频 DApp 成本论证

表 2 Mechanism-Oriented Comparison（机制语义对比）

表 2 的作用是界定机制位置：已有系统通常已经提供交互式挑战或 TEE 快速路径，本文补强的是 timeout 后的 challenge liveness、compact commit 到 DA evidence 的绑定，以及高频 workload 下的成本解释边界。

System	Challenge role	Recovery semantics	DA evidence binding	Cost model role
Optimism	Fraud proof / dispute game	Timeout handled by	Not the focus of this	Not modeled here

		challenge window assumptions	comparison	
Arbitrum	Interactive dispute localization	Protocol-specific dispute progression	Not the focus of this comparison	Not modeled here
OTR	TEE-assisted optimistic verification	Limited timeout recovery discussion	Partial payload/proof linkage	Not modeled here
TEERollup	TEE fast path with challenge entry	Discussed mainly as system mechanism	Discussed at architecture level	No lightweight DA amortization model
This Work	Recoverable challenge state machine	Recover as liveness-preserving transition	Commit-to-DA field proof linkage	Payload/batch/route tradeoff analysis

2 Background and Threat Model

2.1 Rollups, TEE, and Data Availability

Rollup 的基本思想是将大量执行过程移至链下完成，并将状态承诺、证明或争议入口提交到链上。ZK-Rollup 通过有效性证明保证提交状态的正确性，具有较强的验证属性，但证明生成、验证电路设计和复杂系统实现带来较高成本。Optimistic Rollup 假设链下执行结果默认正确，只有在挑战期内出现争议时才进入 fraud proof 或交互式验证流程，因此常规路径成本较低，但结算延迟和挑战机制复杂度成为关键限制。

TEE attestation 为链下执行可信性提供补充接口。可信执行环境可以在隔离环境中执行程序，并对输入、输出、环境标识和 nonce 生成证明，使外部验证者能够确认某一输出来自指定执行环境。然而，TEE 更适合作为降低正常路径验证成本的组件，而不应替代链上挑战、数据可用性和故障追责机制。

Optimistic challenge 是 Rollup 系统中处理异常的重要机制。典型流程包括提交状态承诺、开启挑战、交互式缩小争议范围，并在必要时执行单步重放或链上仲裁。对于长执行轨迹，二分定位能够将争议定位复杂度从线性扫描降低到对数轮次；但 challenge 本身也可能遭遇 timeout、应答缺失或证据不可用，因此其活性和恢复性需要被纳入协议设计。

Data Availability 关注链下或链外数据是否可被需要的验证者获得。模块化区块链和 LazyLedger 等路线强调将数据可用性从执行验证中解耦，使共识层主要负责数据排序和可用性保证。对于 Hybrid TEE-Rollup，DA 的作用不仅是降低存储成本，还要支撑后续 challenge 的证据回溯。因此，compact commit 必须与 DA proof 建立可验证联系。

2.2 Problem Scope and Adversarial Assumptions

本文关注的问题可以概括为：在高频 DApp 场景中，如何让链下执行结果以较低链上成本提交，同时在异常发生时仍能被公开验证、恢复推进并完成仲裁。该问题同时包含成本、可验证性和活性三个维度。成本维度要求链上提交不随完整 payload 线性增长；可验证性维度要求 compact commit 能指向可检查的 DA payload 与证明；活性维度要求 challenge 在 timeout 后不永久停滞。

攻击面覆盖提交者、执行环境、DA 绑定和挑战流程。本文假设执行者或提交者可能提交错误 response、篡改 DA payload、提供无效 attestation，或在挑战流程中延迟响应；DA 层可能出现 payload 缺失、Merkle proof 损坏或 root 不一致；挑战流程本身也可能因超时、中断或 trace 长度不一致而无法自然进入 resolve 阶段。验证者、挑战者或 watchdog 可以观察链上提交和 DA 证据，并在发现异常时发起或恢复挑战。

本文不覆盖真实硬件 TEE 的侧信道攻击、远程证明供应链、真实 DA 网络的经济安全、主网 MEV 或拥堵环境下的费用波动，也不证明完整 Rollup 安全性。这些边界在第 8 节集中讨论；在此处，它们定义了本文协议与实验主张的适用范围。

3 System Overview

系统由执行层、数据层和验证层组成，端到端流程是链下执行、链上留痕、异常时可恢复验证。运行示例贯穿如下：一个高频 DApp 请求进入 simulated TEE，执行层生成 response 与 attestation；数据层保存完整 payload 与字段级证明；链上只接收 compact commit；如果观察者发现 response、attestation 或 DA proof 与提交不一致，则验证层打开 challenge，并在 timeout 后通过 recover 将会话推回 replay/resolve 路径。

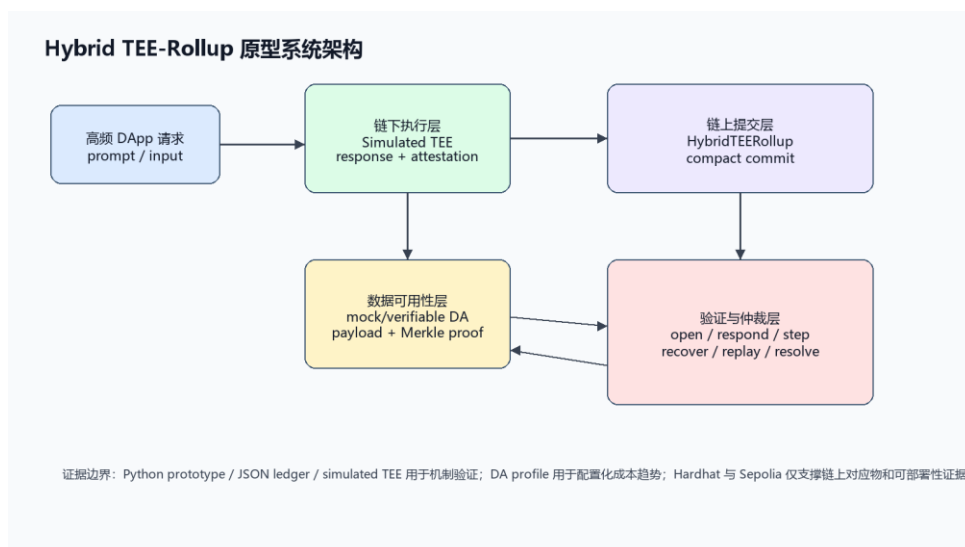


图 1 Hybrid TEE-Rollup 原型系统架构

如图 1 所示，系统由高频 DApp 请求入口、链下执行层、数据可用性层、链上提交层和验证仲裁层组成。正常路径中，执行层输出 response 与 attestation，数据层保存完整证据，链上提交层仅保存 compact commit。异常路径中，验证仲裁层通过 DA pointer 取回证据，并以

challenge、recover、replay 和 resolve 推进状态。这个划分使低成本正常路径与可仲裁异常路径保持同一证据上下文。

3.1 Execution Layer and Normal Path

执行层负责处理高频 DApp 请求。当前原型采用 deterministic mock model 或可选模型后端生成响应，并使用 simulated TEE attestation 对输入哈希、输出哈希、nonce 和执行环境标识进行绑定。attestation 的作用是为链下执行结果提供最小可信摘要，使后续验证层能够检查提交结果是否与声明的执行过程一致。

在系统叙事中，simulated TEE 的角色是建立“链下执行摘要 - 链上提交 - 异常挑战”的接口关系。真实 TEE 安全性不在本节展开，而在 Discussion and Limitations 中集中限定。

3.2 Data Availability Interface

数据层保存完整 payload 及其可验证结构，包括 prompt、response、attestation、payload hash、leaf hashes、Merkle root 和字段级 Merkle proof。系统不将完整 payload 全量上链，而是通过 DA pointer 和 DA root 让链上 compact commit 能够回指证据。第 5 节进一步说明为什么这一提交不是普通 hash，而是后续 replay/challenge 的 evidence entry。

在运行示例中，如果执行者提交的 response 被替换，验证者可从 DA entry 取回 response 字段和对应 Merkle proof，并与链上 da_merkle_root 检查一致性；如果 payload 缺失，则异常停留在 DA availability fault，而不是被误写成已完成 replay 的执行错误。

3.3 Verification Layer and Recovery Trigger

验证层负责处理异常路径。当前原型实现了 challenge open、respond、step、recover、replay 和 resolve 等状态推进。其目标不是将所有执行都搬到链上，而是在出现争议时，通过证据加载、二分定位、单步重放和仲裁结算，将异常状态推进为可解释的结果。

验证层同时承担 fault taxonomy 的职责。本文将故障细分为 attestation fault、DA fault、timeout fault、replay fault 和 trace inconsistency。不同故障对应不同的观察和处理路径。例如，DA unavailable 表明数据层无法提供 payload；DA proof invalid 表明 payload 与链上 root 不一致；trace length mismatch 表明执行轨迹与声明结构不一致；response tampering 则可能同时导致 payload hash、attestation 和 state root mismatch。

在 Solidity 映射中，原型包含 MockTEVerifier、MockDARegistry、MerkleVerifier 和 HybridTEERollup 等合约，用于表达链上状态推进和最小验证逻辑。本地 EVM 合约并非 Python 原型的逐行移植，而是抽取适合链上表达的提交、验证、挑战、恢复和结算路径。

4 Recoverable Challenge Protocol

4.1 Protocol Model and Definitions

本节先给出 challenge session、状态机与 recovery function 的最小形式化接口，再解释 timeout -> recover -> replay 的协议连续性。Formal definitions 不作为独立章节出现，因为它们的作用是约束恢复操作不会改写争议事实，而不是替代后续协议流程。

Definition 1 (Challenge Session)

挑战会话定义为: $cs = (id, s, F, T)$ 。

其中, id 表示会话标识; s 表示当前状态; F 表示 Challenge Facts (挑战事实集合); T 表示超时参数。

Definition 2 (Challenge State Machine)

挑战状态机定义为: $M = (S, E, \delta)$ 。

其中, S 为状态集合; E 为事件集合; $\delta: S \times E \rightarrow S$ 为状态转移函数。

Definition 3 (Recovery Function)

恢复函数定义为: $Recover(cs) = cs'$ 。

恢复仅改变活性相关状态, 不改变 Challenge Facts, 即满足: $F(cs) = F(Recover(cs))$ 。

Protocol Property

基于上述定义, $recover$ 是活性层变换而非正确性裁决: $Recover$ 允许会话从 $TIMED_OUT$ 等停滞状态回到可推进状态, 但不重写 $commit$ 、 $DA\ root$ 、 $payload\ hash$ 、 $trace$ 或 $mismatch\ evidence$ 。该边界把恢复语义与 $slashing$ 语义分开, 也解释了为什么恢复不会改变 $correctness$: 后续 $replay$ 仍然基于原始 Challenge Facts。

4.2 Timeout, Recovery, Replay, and Resolution

Recoverable Challenge Protocol 的目标, 是将 challenge 从一次性错误检测流程扩展为异常场景下可恢复推进的协议状态机。其基本状态包括 $PENDING$ 、 $OPEN$ 、 $RESPONDED$ 、 $NARROWING$ 、 $RECOVERED$ 、 $READY_FOR_REPLAY$ 、 $REPLAYED$ 、 $RESOLVED$ 、 $SLASHED$ 和 $FINALIZED$ 。

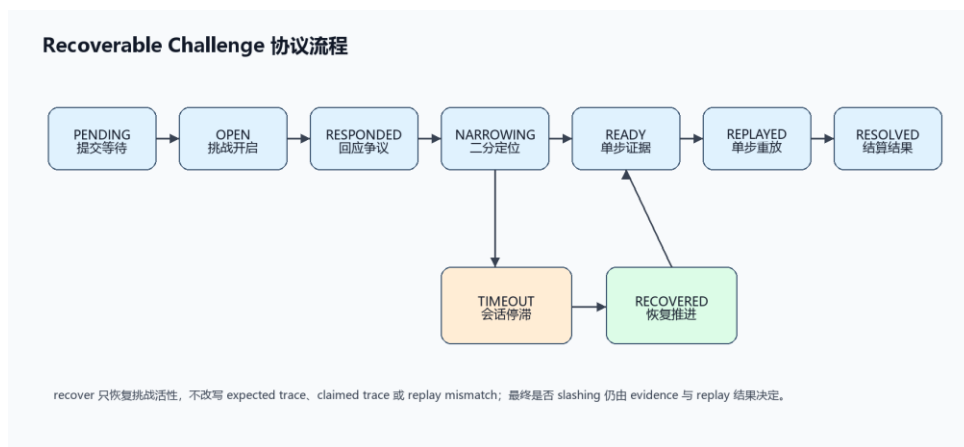


图 2 Recoverable Challenge 协议流程

当交易处于 $PENDING$ 状态时, 挑战者可以调用 $open$ 进入 $OPEN$ 状态。系统加载 $compact\ commit$ 、 $DA\ status$ 和相关 $mismatch\ evidence$ 。随后, 被挑战方进入 $respond$ 阶段, 提供其对争议的回应。若争议涉及执行轨迹, 协议进入 $step$ 阶段, 通过 $bisection\ narrowing$ 缩小争议

区间。对于长度为 n 的 `trace`，理想情况下定位轮次接近 $\log_2(n)$ ，从而避免线性扫描长执行轨迹。

当争议区间被缩小到单步时，协议进入 `single-step replay`。`replay` 阶段比较 `expected hash` 与 `claimed hash`，判断局部执行是否一致。如果重放失败，`resolve` 阶段可以将交易标记为 `SLASHED`；如果重放通过，则挑战不成立或进入相应结算逻辑。

`recover` 是本文强调的关键机制。在普通 `challenge` 中，如果 `session` 超时，流程可能停滞在中间状态，导致系统只能检测异常而无法完成仲裁。`Recoverable challenge` 引入 `watchdog/operator` 恢复路径，使超时 `session` 能够被恢复并继续 `step`、`replay` 和 `resolve`。该机制的价值是 `liveness`：它使 `challenge` 能够从“异常被发现”推进到“异常被处理完”。同时，`recover` 不替换 `expected trace`，不覆盖 `claimed trace`，也不直接决定 `slashing`；它只是把状态机从 `timeout` 后的停滞状态恢复到可继续仲裁的状态。

协议流程可分为三条路径。第一是正常路径：若挑战期内没有有效争议，交易最终进入 `FINALIZED`。第二是可仲裁异常路径：若 `response tampered` 或 `trace length mismatch` 等执行相关故障出现，协议通过 `open`、`respond`、`step`、`replay` 与 `resolve` 完成争议定位和结算。第三是 `timeout` 恢复路径：若挑战过程因超时中断，`watchdog` 或 `operator` 可以调用 `recover` 将会话推进到 `RECOVERED`，并继续后续 `replay` 与 `resolve`。该划分避免将“异常检测”“活性恢复”和“经济惩罚”混写为同一性质。

4.3 Correctness Boundary and Protocol Properties

性质一，DA proof 绑定性质。`compact commit` 中的 `da_merkle_root` 与 DA entry 中的字段级 Merkle proof 绑定，因此 `prompt`、`response` 或 `attestation` 字段被替换后，会导致 `payload hash`、`leaf hash` 或 Merkle proof 与链上 `root` 不一致。该性质不依赖真实 DA 网络，只说明当前 `verifiable DA` 结构能检测字段篡改。

性质二，`recover` 的活性性质。`recover` 操作的目标是恢复超时挑战会话，使其继续进入 `step`、`replay` 和 `resolve`；它不修改 `expected trace`、`claimed trace` 或 `replay mismatch` 的事实。因此，`recover` 不改变争议判断本身，而是避免 `timeout` 让争议流程永久停在中间状态。

性质三，二分定位性质。对于长度为 n 的执行 `trace`，`challenge-step` 每轮缩小争议区间，理想轮次接近 $\log_2(n)$ 。本文实验中 `trace steps` 为 4、8、16、32、64、128 时，平均二分轮次分别为 2、3、4、5、6、7，与理论对数趋势一致。

5 Evidence-Carrying Compact Commit and DA Binding

`Compact commit` 的研究价值不在于把 `payload` 简单压缩成一个 `hash`，而在于把正常路径的低成本提交转换成异常路径可使用的 `evidence entry`。提交结构绑定 `state_root`、`output_hash`、`proof_hash`、`da_pointer` 和 `da_merkle_root`：`state_root` 约束输入、输出和执行环境摘要；`output_hash` 约束 `response`；`proof_hash` 约束 `attestation`；`da_pointer` 定位完整 `payload`；`da_merkle_root` 则为字段级 `proof` 提供链上根承诺。

这种绑定关系使验证者能够从链上提交进入 DA 证据结构。若 response 被篡改，验证者检查 response leaf 与 Merkle root；若 attestation 与输出不一致，proof_hash 和 payload hash 形成交叉约束；若 DA proof 损坏，系统可把异常定位为 DA proof invalid，而不是混同为执行重放失败。由此，compact commit 支持后续 challenge/replay 的证据加载，而不是只服务于链上字节压缩。

从对抗性角度看，提交者可能尝试替换 payload、隐藏 DA entry、提交与 TEE attestation 不一致的 output，或在争议过程中延迟回应。evidence-carrying commit 不能单独解决真实 DA 网络可用性或真实 TEE 证明安全，但它明确了链上最小承诺必须绑定哪些证据字段，才能让异常路径具备可检查入口。

6 DA-aware Cost Model

成本模型关注 full-onchain 与 compact commit + DA 在高频场景下的结构性差异。full-onchain 路线将完整 payload 作为链上数据提交，其成本随 payload size 增大近似线性增长。compact commit 路线只将固定摘要结构提交到链上，将完整 payload 放入 DA 层，并通过 DA root 和 Merkle proof 保留验证能力。

本文比较四类 DA route：full-onchain calldata、compact external DA、compact EIP-4844-like 和 compact modular DA sampling。成本分析是系统权衡问题，而不是单变量压缩问题：payload size 决定 full payload 上链成本增长速度；batch size 决定固定提交开销能否被摊薄；DA route 决定 payload 离开 calldata 后的边际成本。为避免将模型趋势误写为真实价格，本文将该模型称为 lightweight DA cost model：它保留与研究问题直接相关的提交字节、DA profile 和 batch amortization，而不建模实时 gas market、blob base fee、跨域消息费用、主网拥堵或 DA 网络经济安全。

模型表达为 $C_{total} = C_{commit} + C_{DA} + p_{challenge} * C_{challenge}$ 。其中 C_{commit} 表示链上 compact commit 或 full payload 提交成本， C_{DA} 表示不同 DA route 下的数据成本， $C_{challenge}$ 表示发生争议时的挑战、恢复、重放和结算成本， $p_{challenge}$ 表示争议发生概率。批处理后单笔摊销成本为 $C_{amortized} = C_{total} / batch_size$ 。当前实验主要比较不同 route 在相同参数下的结构性趋势，而不估计真实业务中的 $p_{challenge}$ 。

在系统解释上，batch size 越大，固定提交成本越容易被摊薄；payload 越大，full-onchain calldata 的线性成本越突出；DA route 越接近低边际数据成本，compact commit + DA 的收益越明显。当前 Python prototype 中的成本实验是配置化估算模型，本地 EVM gasUsed 则用于补充链上状态推进基线，两者在 Evaluation 中明确区分。

7 Evaluation

Evaluation 围绕三个目标展开。第一，验证 compact commit 是否把链上提交规模与完整 payload 解耦，并保留可追溯证据入口。第二，解释 DA route、payload size 与 batch size 如何共同影响高频 workload 的摊销成本。第三，检验 timeout、trace mismatch 和 response tampering 等对抗性场景下，recoverable challenge 是否能把异常从 detection 推进到 replay、resolve 与 slashing。原型限制也必须提前说明：Python 实验用于机制和趋势，本地 EVM 用于

合约路径基线，Sepolia 用于 deployability evidence；三者不能替代真实 TEE、真实 DA 或主网压力测试。

本文成本实验使用如下抽象模型：

$$C_{\text{total}} = C_{\text{commit}} + C_{\text{DA}} + p_{\text{challenge}} * C_{\text{challenge}}$$

$$C_{\text{amortized}} = C_{\text{total}} / \text{batch_size}$$

其中， C_{commit} 表示链上提交 compact commit 或 full payload 的成本， C_{DA} 表示 DA route 对完整 payload 的数据成本， $C_{\text{challenge}}$ 表示发生争议时 challenge、recover、replay 与 resolve 的成本， $p_{\text{challenge}}$ 表示争议发生概率。当前实验主要比较无争议常规路径下不同提交策略的结构趋势，因此未估计真实业务中的 $p_{\text{challenge}}$ 。DA profile 参数来自实验脚本配置：full_onchain_calldata 的 on-chain gas per byte 为 16.0，DA gas per byte 为 0；compact_external_da 的 on-chain gas per byte 为 16.0，DA gas per byte 为 2.2；compact_eip4844_like 的 on-chain gas per byte 为 16.0，DA gas per byte 为 0.9；compact_modular_da_sampling 的 on-chain gas per byte 为 16.0，DA gas per byte 为 0.45。这些参数仅用于配置化估算，不代表以太坊主网、blob 市场或任何真实 DA 网络的实时价格。

7.1 RQ1: Does compact commit reduce on-chain submission size?

RQ1 的实验从提交结构本身入手。Python prototype 在 JSON ledger 与 simulated TEE 环境中生成提交样本，比较 full payload 与 compact commit 的 UTF-8 canonical JSON 字节数。参数覆盖 payload size 128、512、2048、8192 与 prompt length 64、256，每组参数生成 100 个样本。这里关心的不是主网交易费用，而是提交结构是否已经把链上字节规模从业务 payload 中剥离出来。

度量指标包括 full_bytes、compact_bytes 和 $\text{byte_reduction_ratio} = (\text{full_bytes} - \text{compact_bytes}) / \text{full_bytes}$ 。full_bytes 表示完整 prompt、response、attestation 及相关字段都作为 payload 提交时的字节规模；compact_bytes 则只统计 state_root、output_hash、proof_hash、da_pointer 与 da_merkle_root 等摘要字段。

实验现象比较直接：compact commit 的平均规模约为 406 bytes，并且基本不随 payload size 增长；full payload bytes 则随着 payload size 增大近似线性上升。当 payload size 从 128 增至 8192 时，full bytes 从约 1682.84 / 1876.84 增至约 50068.84 / 50262.84，而 compact bytes 仍停留在固定摘要结构附近。这个数量级差异来自两类提交的结构差异：full payload 把业务内容本身带入链上提交，compact commit 只保留状态、输出、证明和 DA 指针的绑定关系。

RQ1 的结论是有限但关键的：在当前原型的数据结构下，evidence-carrying compact commit 能够把链上提交规模与完整业务 payload 解耦。它没有消除 payload，而是把 payload 移到 DA 层，并通过 root 与 proof 保留后续验证入口。

这一结论的解释边界是编码层面的：字节数来自 Python prototype 的 canonical JSON 表达，不能直接等价于真实链上 calldata、storage layout 或 ABI 编码后的完整成本；真实 attestation 与 DA 证明大小也会改变绝对数值。

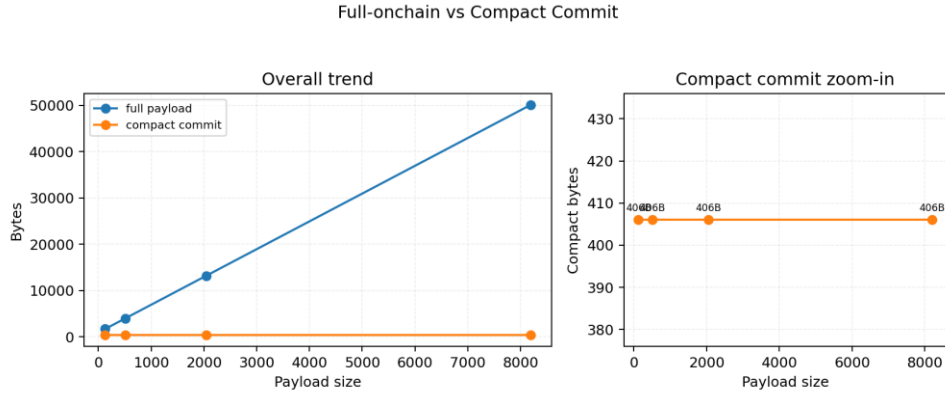


图 3 full payload 与 compact commit 的字节规模对比

7.2 RQ2: How does batch size affect DA-aware amortized cost?

RQ2 关注批处理后的成本摊销。实验使用 DA-aware cost model 比较 full_onchain_calldata、compact_external_da、compact_eip4844_like 和 compact_modular_da_sampling 四类路线，参数覆盖 payload size 128、512、2048、8192，prompt length 64、256，batch size 1、10、100、1000，每组 100 个样本。与 RQ1 只看字节规模不同，RQ2 把固定提交开销、DA route 和 batch size 放在同一个模型中观察。

主要指标为 $\text{amortized_gas} = \text{total_gas} / \text{batch_size}$ 和 $\text{reduction_ratio} = (\text{full_onchain_gas} - \text{target_gas}) / \text{full_onchain_gas}$ 。total_gas 来自配置化 profile，而不是主网交易回执；因此这里讨论的是结构性趋势，而不是实时网络价格。

随着 batch size 增大，各 DA profile 下的单笔摊销成本都会下降。以 payload=8192、prompt length=64 为例，full_onchain_calldata 的单笔估算 gas 从 batch size=1 时的 801101.44 下降到 batch size=1000 时的 801.10；compact_modular_da_sampling 从 29026.98 下降到 29.03。相对于 full_onchain_calldata，compact_modular_da_sampling 在 batch size=1000 时的 reduction_ratio 为 96.33%。这个下降幅度并不神秘：batch size 直接摊薄固定提交成本，而 modular DA sampling profile 又降低了完整 payload 的边际数据成本。

从预期关系看，结果与成本模型一致。payload 越大，full-onchain calldata 需要承担的线性数据成本越明显；batch 越大，固定摘要提交越容易被摊薄；DA route 的差异则决定 payload 离开 calldata 后还能节省多少边际成本。对高频 DApp 来说，真正重要的是这三者的组合，而不是单独某一个参数。

96.33% 是给定 cost model 与 DA profile 参数下的趋势结果。它说明 compact modular DA sampling 在大 payload 与大 batch 条件下具有结构性优势，但真实公网费用仍会受到 calldata 定价、blob 价格、交易拥堵、DA 网络实现和业务争议率影响。

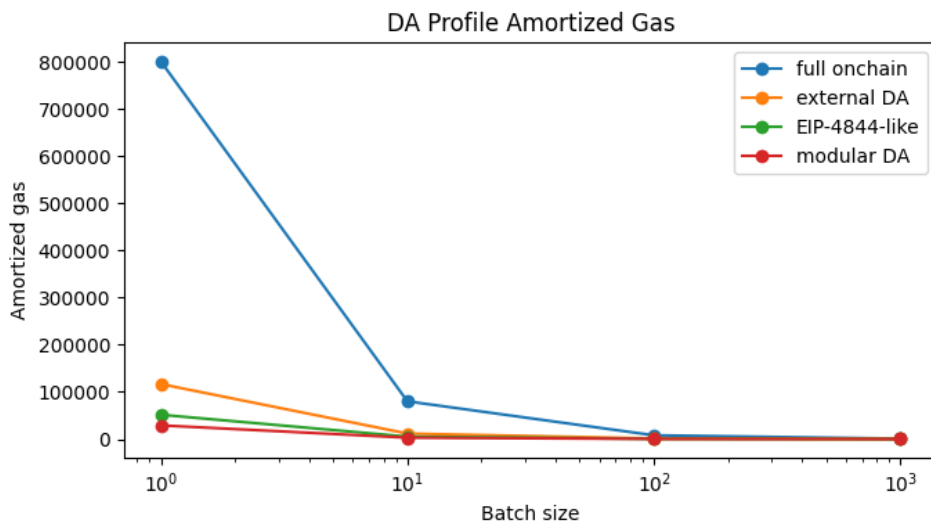


图 4 不同 DA profile 下的摊销 gas 趋势

7.3 RQ3: Can recovery improve challenge liveness under timeout?

RQ3 将注意力转到异常路径。实验使用 Python prototype 的 challenge 状态机，覆盖 trace steps 4、8、16、32、64、128，challenge timeout 设为 2 秒，max bisection rounds 设为 16。对照组为 challenge_timeout_no_recover，实验组为 challenge_timeout_recover，核心问题是 timeout 被检测后，challenge 是否还能继续推进到 replay、resolve 与 slashing。

这里记录 avg_bisection_rounds、avg_expected_log2_rounds、recovery_success_rate、challenge_success_rate 和 slashed_rate。avg_bisection_rounds 用来检查争议定位复杂度是否接近理论预期，challenge_success_rate 与 slashed_rate 则反映仲裁是否真正走到完成状态。

在 trace steps 为 4、8、16、32、64、128 时，平均二分轮次分别为 2、3、4、5、6、7，与 $\log_2(\text{trace steps})$ 基本一致。timeout 对照更能说明 recover 的作用：challenge_timeout_no_recover 场景下 detection rate 为 100%，但 challenge success 为 0%，slashed 为 0%；challenge_timeout_recover 场景下 detection rate、recovery success、challenge success 和 slashed 均为 100%。也就是说，没有 recover 时，系统已经知道异常存在，却不能自然完成仲裁；加入 recover 后，同一类异常可以继续进入 step、replay 与 resolve。

这一现象说明 recover 的价值主要是活性而非正确性裁决。它没有改变 expected trace、claimed trace 或 replay mismatch 的事实内容，而是为已经停滞的 challenge session 提供继续推进的路径。对高频 DApp 来说，这一区别很关键：异常能被发现只是第一步，异常能被处理完才影响系统是否可用。

当前实验中的 challenge trace、expected hash、claimed hash 和部分 replay evidence 由原型逻辑或链下流程提供。因此，RQ3 支撑的是 recoverable state machine 的活性主张，而不是完整链上 fraud proof、FPVM 或真实 ML/TEE 执行验证的安全性。

7.3.1 Recovery Overhead Analysis (恢复开销分析)

利用表 4（原表 3）中已有本地 EVM 数据，challenge step 的平均 gasUsed 为 93807，challenge recover 为 156891。由此可得恢复相对单轮 challenge step 的开销比约为：Recovery Overhead = $156891 / 93807 \approx 1.67$ 。

Recover 的单次链上开销确实高于一次 challenge step，但它处在低频异常路径上。其作用是在 timeout 已被检测、流程却停滞时恢复 challenge liveness，使会话继续进入 replay 与 resolve。对于高频 DApp 的正常吞吐路径，这一额外成本通常不会进入每笔请求的常规开销；更合理的理解是，它用一次较高的异常路径成本换取争议流程不永久卡死。

上述数值来自 Hardhat 本地 EVM 原型测量，用于解释异常路径的相对成本；主网 gas 价格、拥堵条件和生产级合约优化不在该测量范围内。

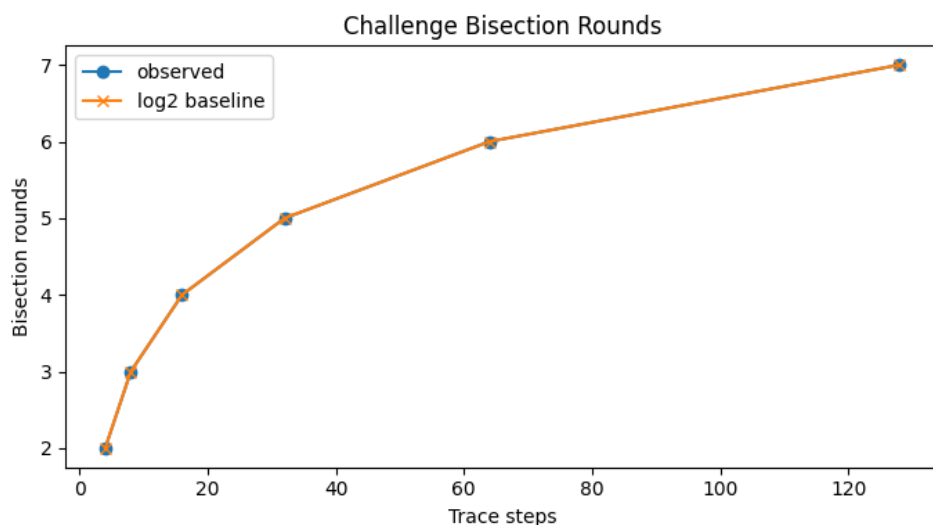


图 5 二分挑战轮次与 trace steps 的关系

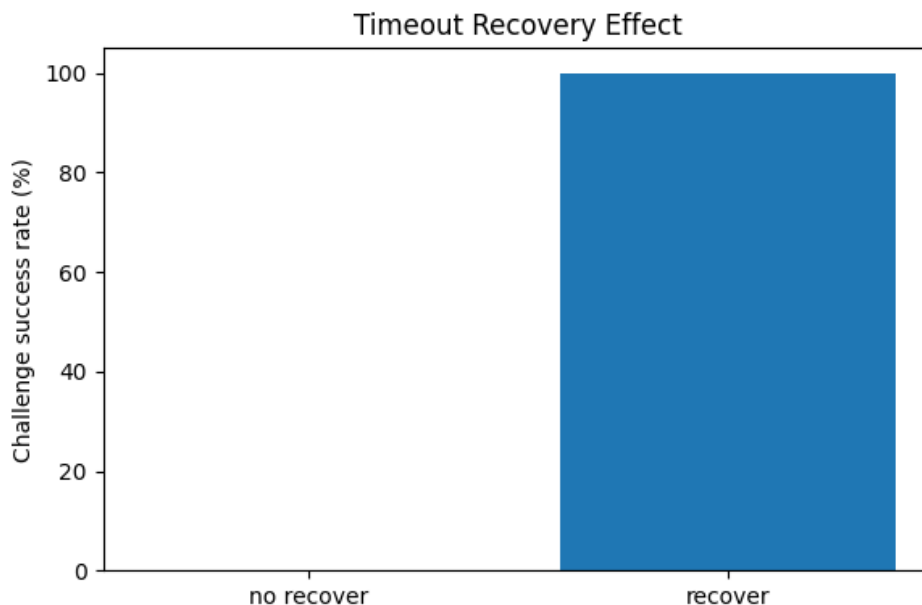


图 6 timeout recover 对挑战完成率的影响

7.4 RQ4: Which failure scenarios lead to detection, recovery, or slashing?

RQ4 进一步拆开 fault taxonomy。failure scenario 实验覆盖 8 类场景，每类 100 个样本，共 800 条记录，包括 normal、attestation_invalid、da_unavailable、da_proof_invalid、response_tampered、trace_length_mismatch、challenge_timeout_no_recover 和 challenge_timeout_recover。这里不把所有异常都强行归为同一种处理结果，而是观察哪些故障在当前原型中已经进入 challenge + slashing，哪些还停留在检测、拒绝或报告层面。

指标为 detection_rate、challenge_success_rate 和 slashed_rate。为了避免把语义不同的异常混在一起，实验分析分为四类：Normal path, Detectable but not unified slashing, Challenge + slashing, 以及 Timeout liveness comparison。

Normal path 中，normal 不触发异常检测或 slashing。Detectable but not unified slashing 类包括 attestation_invalid、da_unavailable 和 da_proof_invalid，这些场景检测率均为 100%，但 challenge success 与 slashed 均为 0%，说明当前原型主要实现检测或拒绝路径。Challenge + slashing 类包括 response_tampered 和 trace_length_mismatch，相关场景能够进入挑战与惩罚闭环。Timeout liveness comparison 中，challenge_timeout_no_recover 能检测 timeout，却无法完成 challenge；challenge_timeout_recover 则能够恢复并推进到 slashing。这个分层现象与系统设计相符：响应篡改和 trace 不一致具备可进入 replay/resolve 的争议材料，而 DA unavailable、DA proof invalid 或 attestation invalid 在当前版本中还没有被统一映射为经济惩罚。

RQ4 的重点不是宣称所有 fault 已经闭环，而是明确当前闭环在哪里。响应篡改、trace 不一致和 timeout recovery 已形成较完整的 challenge + slashing 路径；attestation fault 与 DA fault 则主要体现为稳定检测、拒绝或报告能力。这个区分使 evaluation 更接近

systems/security 论文中的 threat-specific reasoning，而不是把“能发现异常”误写成“都能惩罚异常”。

后续仍需细化 fault taxonomy。特别是 DA unavailable、DA proof invalid 和 attestation invalid，应进一步区分何时直接拒绝、何时触发恢复、何时进入链上仲裁，以及何时应施加经济惩罚。

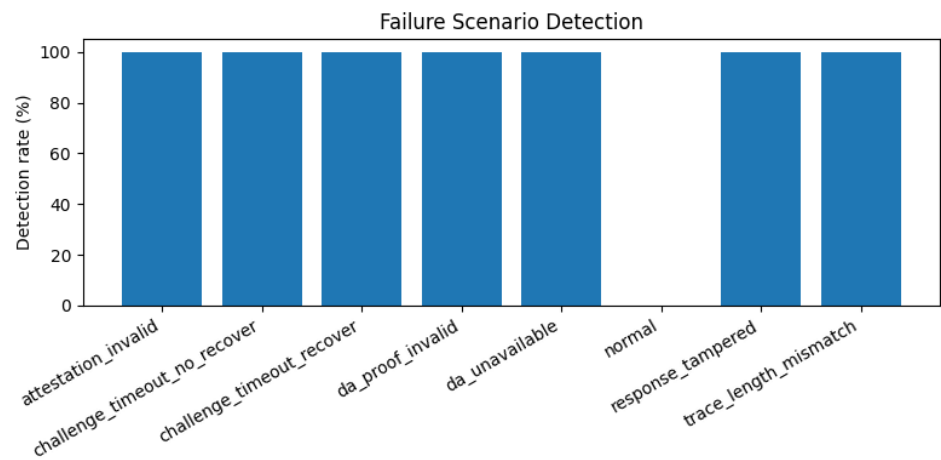


图 7 失败场景检测与仲裁结果

表 3 失败场景分类与当前证据边界

类别	场景	当前结果	证据边界
Normal path	normal	不触发检测或 slashing	说明正常路径不误报
Detectable but not unified slashing	attestation_invalid, da_unavailable, da_proof_invalid	检测率 100%, challenge success/slashed 为 0%	说明可检测或拒绝，不说明已统一惩罚
Challenge + slashing	response_tampered, trace_length_mismatch	detection/challenge success/slashed 均为 100%	说明典型执行异常可进入闭环
Timeout liveness comparison	challenge_timeout_no_recover vs challenge_timeout_recover	no recover 为 0%, recover 后为 100%	说明 recover 改善仲裁活性

7.5 RQ5: Do Solidity, local EVM, and Sepolia evidence support an on-chain counterpart?

RQ5 回到链上对应物。实验使用 Solidity 原型和 Hardhat 本地测试链测量关键操作 gasUsed。Solidity 原型包含 MockTEEVerifier、MockDARegistry、MerkleVerifier 和 HybridTEERollup；测量脚本覆盖 payload size 128、512、2048，并记录 register DA、submit rollup、challenge open、respond、step、recover、replay、resolve 和 finalize 的 gasUsed。

本地 EVM 指标是各关键函数调用的 gasUsed。Sepolia 证据包括 chain id、部署区块、合约地址和 ABI/address index。前者用于观察链上状态推进路径的基线开销，后者用于说明同一组核心合约已经具备公开测试网 deployability evidence。

本地 EVM 平均 gasUsed 包括: register DA 113526.00, submit rollup 299033.67, challenge open 283769.00, challenge respond 65886.00, challenge step 93807.00, challenge recover 156891.00, challenge replay 135632.00, challenge resolve 113402.00, finalize 33477.00。Sepolia 部署记录则给出了公开测试网链上地址与 ABI/address index。将这两类证据放在一起看,可以说明 Python prototype 中的提交、挑战、恢复、重放和结算路径已经被抽取成 Solidity 状态推进操作,但还不能说明它们已经经历生产网络压力测试。

因此, RQ5 支撑的是一个阶段性主张: 当前机制具有链上对应物。本地 Hardhat 测量给出了关键路径 gasUsed 基线, Sepolia 部署说明核心合约可以部署到公开测试网。这个主张的范围很窄,但它补上了从 Python 状态机到链上合约表达之间的一步。

本地 EVM gasUsed 不能等价于公网费用, Sepolia 部署也不能说明生产可用性、主网性能或经济安全。当前 Sepolia 证据仅支持 deployability evidence。

表 4 本地 EVM 关键路径平均 gasUsed

操作	平均 gasUsed	解释
register DA	113526.00	MockDARegistry 记录 payload hash 与 DA root
submit rollup	299033.67	compact commit 提交路径
challenge open	283769.00	打开争议并加载证据
challenge respond	65886.00	被挑战方响应
challenge step	93807.00	二分定位的单轮推进
challenge recover	156891.00	恢复超时挑战会话
challenge replay	135632.00	单步重放仲裁
challenge resolve	113402.00	结算争议结果
finalize	33477.00	挑战期后最终确认

表 5 Sepolia 部署信息

项目	数值
Network / Chain ID	Sepolia / 11155111
部署区块	10825904
MockTEEVERifier	0xC42CB0Cf0D112Bd59E0f212F2DB2002ca50a8b6A
MockDARegistry	0xc42C64a7De05bf3ad4f1c75CbD8E0506F7De99Ff
HybridTEERollup	0xB9B72f10bB8aBC1ed090f1B0443Fe57189c497Fd

7.6 Reproducibility and Evidence Types

Python 原型位于 project_code/tee_rollup_demo, 正式实验脚本为 project_code/scripts/run_paper_experiments.py, 输出 cost_grid.csv、challenge_grid.csv、failure_scenarios.csv、summary.json 和 figures 目录。本地 EVM gas 脚本为 project_code/evm/scripts/measure_gas.js, 输出 paper_outputs/evm_gas/measured_gas_report.md 与 measured_gas_summary.json。Sepolia 部署记录位于 project_code/evm/deployments/sepolia.json。

复现实验可使用以下命令：`python -m unittest discover -s tests -v; python scripts/run_paper_experiments.py --clean --samples 100 --payload-sizes 128,512,2048,8192 --batch-sizes 1,10,100,1000 --trace-steps 4,8,16,32,64,128`；在 `project_code/evm` 目录下运行 `npm.cmd test` 和 `npm.cmd run measure:gas`。上述命令分别对应机制验证、配置化成本趋势和本地 EVM 状态推进基线，不能相互替代。

8 Discussion and Limitations

本节合并原 `limitations`、`future work`、`deployment boundary`、`related work` 和 `conclusion` 中的边界讨论。核心观点是：本文的研究价值来自异常路径机制的组合，而不是生产就绪声明。`simulated TEE`、`mock/verifiable DA registry` 和本地 EVM 都限制了外推范围；但这些限制不取消本文的问题价值，因为 `high-frequency Hybrid TEE-Rollup` 仍需要解释轻量提交如何携带证据、`timeout` 后 `challenge` 如何恢复、以及 `DA route` 如何影响成本权衡。

8.1 Internal Validity（内部效度）

内部效度主要受 `simulated TEE`、`mock/verifiable DA registry`、`Python` 原型逻辑与 `Hardhat` 本地 EVM 测量方式影响。原型中的 `attestation` 只建模输入/输出绑定，`DA` 层也没有接入真实采样与经济安全机制，因此实验验证的是机制接口是否能跑通，而不是硬件安全或 `DA` 经济安全。本地 `gasUsed` 还可能受到合约简化、测试数据规模固定和本地链环境的影响。

8.2 External Validity（外部效度）

外部效度的限制来自实验环境到主网或生产网络之间的差距。主网 `MEV`、拥堵费率波动、`blob` 价格变化、真实 `TEE` 供应链攻击、侧信道风险、宿主 `I/O` 操控与真实 `DA` 网络运营风险，均未在当前实验中覆盖。`Sepolia` 部署只能说明核心合约具有公开测试网可部署性，不能据此外推主网性能或安全强度。

8.3 Construct Validity and Deployment Boundary（构念效度与部署边界）

构念效度关注指标与论文主张之间是否匹配。`challenge success`、`recovery_success` 与 `slashed_rate` 衡量的是原型状态机下的仲裁推进，而不是完整 `Rollup` 经济安全；`DA-aware cost model` 是配置化趋势模型，不等价于真实 `calldata/blob` 定价；`fault taxonomy` 也尚未形成统一 `slashing` 闭环。因此，论文结论应理解为机制层研究原型结论，而非生产系统审计结果。

具体而言，`TEE` 部分仍停留在模拟层。`attestation` 用于绑定输入、输出、`nonce` 和执行环境标识，目的在于验证“链下执行摘要 - 链上提交 - 异常挑战”的协议接口；真实 `SGX`、`TDX`、`CSV` 或其他硬件 `TEE` 还需要处理远程证明格式、硬件漏洞、侧信道攻击、宿主 `I/O` 操控和多厂商信任假设。

`DA` 部分同样是 `mock/verifiable DA` 原型。系统实现了 `DA root`、`payload hash`、字段级 `Merkle proof` 和可用性状态记录，但没有接入 `Celestia`、`EigenDA` 或其他真实 `DA` 网络。因而，`compact commit` 与 `verifiable DA structure` 在本文中只能说明可检查证据入口的形成方式，不能等同于生产级 `DA availability`。

成本模型也应按配置化估算理解。DA profile 中的 gas per byte 参数用于构造可比较的趋势模型，而不是主网 calldata、blob 或真实 DA 服务的实时价格。由此得到的 96.33% 降本只对应给定 cost model 下的趋势结果，不能外推为主网级或生产级费用结论。

Hardhat gasUsed 提供的是本地 EVM 基线。它说明 Solidity 原型中的 submit、challenge open、recover、replay、resolve 和 finalize 等关键路径具有链上状态推进对应物，并能被本地测试链测量；但它不等价于 Sepolia 或主网费用，也没有覆盖公网拥堵、交易排序、blob 价格和跨合约部署环境差异。

Sepolia 部署记录属于公开测试网 deployability evidence。部署区块、合约地址和 ABI 索引说明当前合约可以部署到公开测试网；同时，源码验证曾受 block explorer connect timeout 影响，也尚未记录大规模 Sepolia 交互交易样本。该证据不应被解释为生产成熟度、主网性能或完整安全性的证明。

fault taxonomy 仍需继续收敛。当前 response_tampered、trace_length_mismatch 和 challenge_timeout_recover 可以进入 challenge + slashing 路径；attestation_invalid、da_unavailable 和 da_proof_invalid 主要体现为检测、拒绝或报告能力。后续需要明确不同 fault 的仲裁策略，例如哪些故障应直接拒绝，哪些故障需要 watchdog 恢复，哪些故障应进入链上仲裁或经济惩罚。

challenge replay 中的部分 evidence 仍由链下提供。原型通过 expected hash、claimed hash、trace evidence 和 mismatch bits 支持 replay arbitration，但没有实现完整链上可验证执行环境、FPVM 或真实模型推理单步证明。下一步需要加强 evidence 的链上可验证性、形式化安全论证和跨实现一致性测试，减少 replay 结果对链下诚实生成证据的依赖。

综合这些边界，本文仍然具有研究价值：recoverable challenge、evidence-carrying compact commit 和 DA-aware cost model 把异常活性、证据入口和成本权衡放入同一个可运行原型中，使 Hybrid TEE-Rollup 面向高频 DApp 的薄正常路径与可仲裁异常路径可以被共同讨论。未来工作应集中在真实 TEE attestation、真实 DA 网络接入、统一 fault arbitration、链上可验证 replay evidence、形式化安全论证和大规模公网交互测量。

8.4 Relationship to Prior Work

本文与现有工作的关系属于 mechanism enhancement，而非 system replacement。本文并不试图替代 TEERollup、OTR 或 opML 等系统路线，而是在其异常路径上补充 Recoverable Challenge 与 Evidence-Carrying DA Binding，使 Hybrid TEE-Rollup 在 timeout、DA 异常与轻量提交场景下具备更可解释的仲裁推进能力。

TEEROLLUP 提出了利用异构 TEE 降低 Rollup 验证成本并缩短提现延迟的系统设计，其核心包括 TEE 委员会、链上状态合约、挑战机制和数据可用性惩罚。本文受到 Hybrid TEE-Rollup 思路启发，但并不重做完整 TEEROLLUP 系统，而是聚焦异常场景下 challenge 的恢复性，以及 compact commit 与 verifiable DA 的成本和证据结构。换言之，本文的贡献位置不是替代既有 TEE-Rollup 架构，而是在其“TEE 快速路径 + 链上异常路径”的思想下，补充 timeout 后争议会话如何恢复推进、轻量提交如何保留 DA 证据入口、以及成本趋势如何以可复现实验方式表达。

OTR 和 Optimistic TEE-Rollups 关注如何将 TEE 与 optimistic verification 结合，尤其适用于链上生成式 AI 或模型推理结果验证。这类工作表明 TEE 可以作为快速路径，而 optimistic challenge 可作为异常路径。本文与其相近之处在于同样关注 TEE 与 challenge 的结合，差异在于本文强调 recoverable challenge 的 liveness，以及 DA payload 与 compact commit 之间的证据绑定。

opML 等工作研究如何通过 optimistic fraud proof 支持机器学习或大模型计算，通常包含二分定位、单步仲裁和链上虚拟机验证。本文借鉴交互式争议定位思想，但没有实现完整 FPVM 或真实 ML 执行证明，而是将 bisection replay 用于 Hybrid TEE-Rollup 原型中的异常仲裁。

Dynamic Fraud Proof 关注如何缩短无争议场景下的最终性，并在发现争议时动态延迟结算。该方向强调 challenge window 和验证者参与机制的动态调整。本文不提出完整动态最终性协议，而是在更小范围内研究 timeout 后 challenge session 如何恢复推进。

LazyLedger 和 Light Clients for Lazy Blockchains 等工作强调将数据可用性从执行验证中解耦，并通过数据可用性抽样支持模块化扩容。本文没有实现真实 DAS 网络，而是使用 mock DA 和 DA profile 建模 compact commit + verifiable DA 的成本与证据结构。本文的重点不是替代 DA 层，而是说明 Hybrid TEE-Rollup 中链上轻量提交必须与 DA 证明建立可验证联系。

EIP-4844 将 blob-carrying transaction 引入以太坊扩容路线，为 Rollup 数据发布提供了区别于传统 calldata 的成本结构。本文中的 compact EIP-4844-like profile 只抽象了“数据发布成本低于 full calldata”的趋势，并未模拟真实 blob base fee、blob 生命周期或主网交易调度。因此，EIP-4844 在本文中是成本建模参照，而不是已经接入的真实数据通道。

表 6 Related Work 与本文工作的对照

方向 / 工作	核心关注	与本文关系	本文不声称覆盖的部分
TEEROLLUP / Hybrid TEE-Rollup	利用异构 TEE 降低 Rollup 验证成本，并保留挑战和 DA 处罚机制	本文沿用 Hybrid TEE-Rollup 的研究方向，聚焦 recoverable challenge 与 evidence-carrying compact commit	不实现完整 TEE 委员会、生产级提现流程或完整经济安全
Optimistic TEE-Rollups / OTR	将 TEE 快速执行与 optimistic verification 结合	本文同样采用“正常路径低成本、异常路径挑战”的思想	不证明真实 TEE 远程证明安全或链上生成式 AI 生产服务可用性
opML / optimistic ML fraud proof	通过二分定位和单步证明验证 ML 或复杂计算	本文借鉴 bisection replay 的争议定位结构	不实现完整 FPVM、真实模型推理单步证明或链上 ML 执行环境
Dynamic Fraud Proof	动态调整争议窗口和最终性，以缩短无争议路径延迟	本文关注 timeout 后会话恢复，是更窄的 challenge liveness 问题	不提出完整动态最终性协议
LazyLedger / 模块化 DA	将 DA 与执行验证解耦，支持数据可用性抽样	本文用 mock/verifiable DA 与 DA profile 说明 compact commit 需要绑定 DA 证据	不接入真实 DA 网络，也不证明 DAS 经济安全
Light Clients for Lazy Blockchains	面向轻客户端的数据可用性抽样与验证	本文借鉴“轻量验证者需要可检查数据承诺”的思想	不实现真实轻客户端协议
EIP-4844 / blob data	降低 Rollup 数据发布成本，提供 blob 数据市场	本文将 EIP-4844-like 作为配置化成本 profile	不模拟真实 blob base fee、主网拥堵或 blob 生命周期

从对照可以看出，已有工作已经提供 TEE-Rollup、optimistic fraud proof 和 modular DA 的主要路线；本文的课程论文工作把 recoverable challenge、DA-bound compact commit 和轻量成本模型放入同一个可运行研究原型中，并分别用 Python、Hardhat 与 Sepolia 证据说明机制可行性、链上对应物和可部署性边界。

8.5 Takeaways and Future Direction

高频 DApp 的核心困难不只是吞吐量不足，还在于低成本交互与异常可验证之间经常互相拉扯。完全上链会把大量低价值高频请求转化为高昂链上成本；完全链下或单纯依赖 TEE 又会削弱异常时的公开仲裁能力。本文在已有 Hybrid TEE-Rollup 思路下回答的是一个更窄的问题：能否在不重做完整 Rollup 系统的前提下，让轻量提交、DA 证据和可恢复 challenge 形成可运行的研究原型。

实验给出的回答是谨慎肯定的。Python prototype 跑通了提交、DA proof、挑战、恢复、重放和结算的基本闭环；结果中可以观察到 compact commit 的固定成本特征、payload 增大后 DA 路线的收益、batch size 对 amortized gas 的摊薄作用、recover 对 challenge liveness 的改善，以及 bisection 轮次与理论对数复杂度的一致性。本地 EVM 和 Sepolia 部署进一步说明，原型中的关键路径已经能够被抽取为 Solidity 合约操作，并具有公开测试网部署记录。

后续研究可以沿两个方向推进：一是把 evidence 的链上可验证性做得更强，减少 replay 对链下证据生成的依赖；二是在真实 DA 与 TEE 环境中重新测量成本、活性和异常处理边界。这样，Hybrid TEE-Rollup 面向高频 DApp 的可验证低成本交互，才可能从研究原型继续走向更完整的系统评估。

8.5.1 Theoretical Implication（理论启示）

从理论层面看，本文更有价值的部分不是提出新的 Rollup 系统，而是把 Challenge Recovery 从工程补丁提升为具有协议语义的独立机制。Recoverable Challenge、Evidence-Carrying Commit 与 DA-Aware Cost Model 的组合，为 Hybrid TEE-Rollup 在高频 DApp 场景下的异常处理与低成本验证提供了一个可讨论的研究视角。

参考文献

- [8] Intel Corporation. Intel Software Guard Extensions (Intel SGX) Developer Guide[EB/OL]. 2024.
- [9] Intel Corporation. Intel Trust Domain Extensions (Intel TDX) Architecture Specification[EB/OL]. 2024.
- [10] Offchain Labs. Arbitrum: A Next-Generation Layer 2 for Ethereum[EB/OL]. 2021.
- [11] Cartesi. Cartesi Rollups Documentation[EB/OL]. 2024.
- [12] EigenLayer. EigenDA Documentation[EB/OL]. 2024.
- [13] Celestia. Celestia: Modular Blockchain Network[EB/OL]. 2024.
- [14] RISC Zero. RISC Zero zkVM Documentation[EB/OL]. 2024.
- [15] Succinct Labs. SP1: A Performant, 100% Open-Source zkVM[EB/OL]. 2024.
- [16] Arbitrum Foundation. Arbitrum Nitro Technical Documentation[EB/OL]. 2024.

- [17] OP Labs. Optimism Documentation: Fault Proofs and Dispute Games[EB/OL]. 2024.
- [18] Canetti R, et al. Universally Composable Security: A New Paradigm for Cryptographic Protocols[EB/OL]. 2000.
- [19] Goldwasser S, Micali S, Rackoff R. The Knowledge Complexity of Interactive Proof Systems[J]. SIAM Journal on Computing, 1989, 18(1): 186-208.
- [20] NIST. Recommendation for Key Management: Part 1 - General (Rev. 5)[EB/OL]. 2020.
- [1] WEN X, FENG Q, LYU H, et al. TEEROLLUP: Efficient Rollup Design Using Heterogeneous TEE[EB/OL]. arXiv:2409.14647, 2024.
- [2] PICCO G, FORTUGNO A. Dynamic Fraud Proof[EB/OL]. arXiv:2502.10321, 2025.
- [3] AL-BASSAM M. LazyLedger: A Distributed Data Availability Ledger with Client-Side Smart Contracts[EB/OL]. arXiv:1905.09274, 2019.
- [4] TAS E N, TSE D, YANG L, et al. Light Clients for Lazy Blockchains[EB/OL]. arXiv:2203.15968, 2022.
- [5] Ethereum Foundation. EIP-4844: Shard Blob Transactions[EB/OL]. 2024.
- [6] Ethereum Foundation. Optimistic Rollups[EB/OL]. Ethereum Documentation, 2024.
- [7] Buterin V. An Incomplete Guide to Rollups[EB/OL]. 2021.